



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій

Лабораторна робота №4
із дисципліни «**Технології розроблення програмного забезпечення**»
Вступ до паттернів проектування.

Виконав:

студент групи ІА–34

Оборський Дмитро Вікторович

Київ 2025

Тема: Вступ до паттернів проектування.

Мета: Вивчити структуру шаблонів «Singleton», «Iterator», «Proxy», «State», «Strategy» та навчитися застосовувати їх в реалізації програмної системи.

14. Архіватор (strategy, adapter, factory method, facade, visitor, p2p)

Архіватор повинен являти собою візуальний додаток з можливістю створення і редагування архівів різного типу (.tar.gz, .zip, .rar, .ace) – додавання/ видалення файлів / папок, редагування метаданих (по можливості), перевірка checksum архівів, тестування архівів на наявність пошкоджень, розбиття архівів на частини.

Теоретичні відомості

1.1. Теоретичні відомості

1.1.1. Поняття шаблону проектування

Будь-який патерн проектування, використовуваний при розробці інформаційних систем, являє собою формалізований опис, який часто зустрічається в завданнях проектування, вдале рішення даної задачі, а також рекомендації по застосуванню цього рішення в різних ситуаціях [5]. Крім того, патерн проектування обов'язково має загальнозживане найменування. Правильно сформульований патерн проектування дозволяє, відшукавши одного разу вдале рішення, користуватися ним знову і знову. Варто підкреслити, що важливим початковим етапом при роботі з патернами є адекватне моделювання розглянутої предметної області. Це є необхідним як для отримання належним чином формалізованої постановки задачі, так і для вибору відповідних патернів проектування.

Відповідне використання патернів проєктування дає розробнику ряд незаперечних переваг. Наведемо деякі з них. Модель системи, побудована в межах патернів проєктування, фактично є структурованим виокремленням тих елементів і зв'язків, які значимі при вирішенні поставленого завдання. Крім цього, модель, побудована з використанням патернів проєктування, більш проста і наочна у вивченні, ніж стандартна модель. Проте, не дивлячись на простоту і наочність, вона дозволяє глибоко і всебічно опрацювати архітектуру розроблюваної системи з використанням спеціальної мови.

Застосування патернів проєктування підвищує стійкість системи до зміни вимог та спрощує неминуче подальше доопрацювання системи. Крім того, важко переоцінити роль використання патернів при інтеграції інформаційних систем організації. Також слід зазначити, що сукупність патернів проєктування, по суті, являє собою єдиний словник проєктування, який, будучи уніфікованим засобом, незамінний для спілкування розробників один одним.

Таким чином шаблони представляють собою, підтверджені роками розробок в різних компаніях і на різних проєктах, «ескізи» архітектурних рішень, які зручно застосовувати у відповідних обставинах.

1.1.2. Шаблон «Singleton»

Призначення патерну: «Singleton» (Одинак) являє собою клас в термінах ООП, який може мати не більше одного об'єкта (звідси і назва «одинак») [6]. Насправді, кількість об'єктів можна задати (тобто не можна створити більш п об'єктів даного класу). Даний об'єкт найчастіше зберігається як статичне поле в самому класі.

Проблема: Використання одинака виправдано для наступних випадків:

- може бути не більше N фізичних об'єктів, що відображаються в певних класах;
- необхідно жорстко контролювати всі операції, що проходять через даний клас.

Одинак вирішує відразу дві проблеми, порушуючи принцип єдиної відповідальності класу.

Singleton
- instance : Singleton
- Singleton() + Instance() : Singleton

Рисунок 4.1. Структура патерну Одинак

Рішення:

Перший випадок легко продемонструвати. У кожної програми є певний набір налаштувань, який як правило зберігається в окремий файл (для сучасних комп'ютерних ігор це може бути .ini файл, для .net додатків –.xml файл). Цей файл – єдиний, і тому використання безлічі об'єктів для завантаження/запису даних – нераціональне рішення.

Для демонстрації другого випадку, розглянемо наступний приклад. Припустимо, існує дві взаємодіючі системи, між якими встановлено сеанс зв'язку. Накладені обмеження, що по даному сеансу зв'язку дані можуть йти в один момент часу лише в одну сторону. Таким чином, на кожен надісланий запит необхідно дочекатися відповіді, перш ніж відсилати новий запит. Об'єкт «одинак» дозволить не тільки містити рівно один сеанс зв'язку, а й ще реалізувати відповідну логіку перевірок на основі bool операторів про можливість відправки запиту і, можливо, деяку чергу запитів.

Переваги та недоліки: Однак слід зазначити, що в даний час патерн «Одинак» багато хто вважає т.зв. «анти-шаблоном», тобто поганою практикою проєктування. Це пов'язано з тим, що «одинаки» представляють собою глобальні дані (як глобальна змінна), що мають стан. Стан глобальних об'єктів важко відслідковувати і підтримувати коректно; також глобальні об'єкти важко тестуються і вносять складність в програмний код (у всіх ділянках коду виклик

в одне єдине місце з «одинаком»; при зміні підходу доведеться змінювати масу коду).

При цьому реалізація контролю доступу можлива за допомогою статичних змінних, замикань, мютексов та інших спеціальних структур.

- + Гарантує наявність єдиного екземпляра класу.
- + Надає до нього глобальну точку доступу.
- Порушує принцип єдиної відповідальності класу.
- Маскує поганий дизайн.

1.1.3. Шаблон «Iterator»

Призначення: «Iterator» (Ітератор) являє собою шаблон реалізації об'єкта доступу до набору (колекції, агрегату) елементів без розкриття внутрішніх механізмів реалізації. Ітератор виносить функціональність перебору колекції елементів з самої колекції, таким чином досягається розподіл обов'язків: колекція відповідає за зберігання даних, ітератор – за прохід по колекції [6].

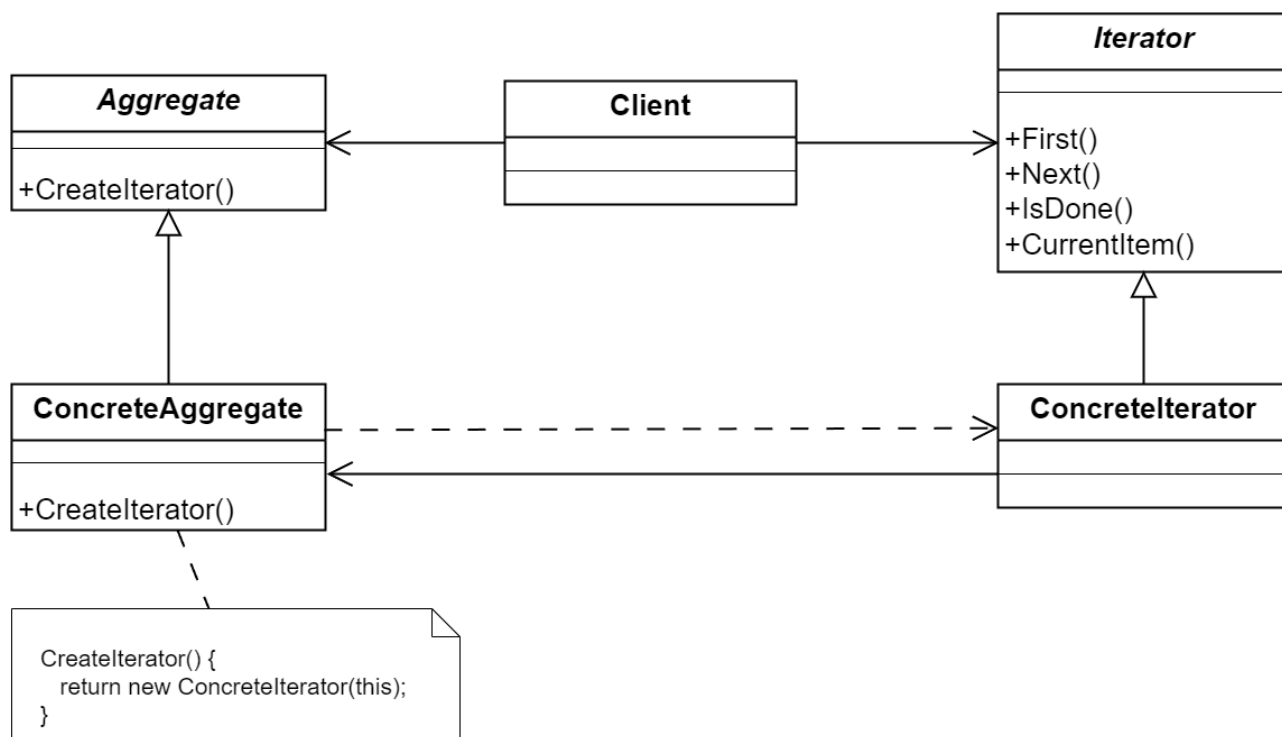


Рисунок 4.2. Структура патерну Ітератор

При цьому алгоритм ітератора може змінюватися – при необхідності пройти в зворотньому порядку використовується інший ітератор. Можливо також написання такого ітератора, який проходить список спочатку по парних позиціях (2,4,6-й елементи і т.д.), потім по непарних. Тобто, шаблон ітератор дозволяє реалізовувати різноманітні способи проходження по колекції незалежно від виду і способу представлення даних в колекції.

Проблема: Більшість колекцій виглядають як звичайний список елементів. Але є й екзотичні колекції, побудовані на основі дерев, графів та інших складних структур даних.

Але як би не була структурована колекція, користувач повинен мати можливість послідовно обходити її елементи, щоб виробляти з ними якісь дії.

Але яким способом слід переміщатися по складній структурі даних? Наприклад, сьогодні може бути достатнім обхід дерева в глибину, але завтра буде потрібно можливість переміщатися по дереву в ширину. А на наступному тижні і того гірше – знадобиться обхід колекції у випадковому порядку.

Додаючи все нові алгоритми в код колекції, ви потроху розмиваєте її основне завдання, яке полягає в ефективному зберіганні даних.

Рішення: Ідея патерна Ітератор полягає в тому, щоб винести поведінку обходу колекції з самої колекції в окремий клас.

Об'єкт-ітератор буде відстежувати стан обходу, поточну позицію в колекції і скільки елементів ще залишилося обійти. Одну і ту ж колекцію зможуть одночасно обходити різні ітератори, а сама колекція не буде навіть знати про це.

До того ж, якщо вам знадобиться додати новий спосіб обходу, ви зможете створити окремий клас ітератора, не змінюючи існуючий код колекції.

Шаблонний ітератор містить:

- First() – установка покажчика перебору на перший елемент колекції;
- Next() – установка покажчика перебору на наступний елемент колекції;
- IsDone – булевське поле, яке встановлюється як true коли покажчик перебору досяг кінця колекції;

- `CurrentItem` – поточний об'єкт колекції.

Переваги та недоліки: Цей шаблон дозволяє уніфікувати операції проходження по наборам об'єктів для всіх наборів. Тобто, незалежно від реалізації (масив, зв'язаний список, незв'язаний список, дерево та ін.), кожен з наборів може використовувати будь-який з реалізованих ітераторів.

- + Дозволяє реалізувати різні способи обходу структури даних.
- + Спрощує класи зберігання даних.
- Не виправданий, якщо можна обійтися простим циклом.

1.1.4. Шаблон «Proxy»

Призначення: «Proxy» (Проксі) – об'єкти є об'єктами-заглушками або двійниками/замінниками для об'єктів конкретного типу. Зазвичай, проксі об'єкти вносять додатковий функціонал або спрощують взаємодію з реальними об'єктами [5].

Проксі об'єкти використовувалися в більш ранніх версіях інтернет-браузерів, наприклад, для відображення картинки: поки картинка завантажується, користувачеві відображається «заглушка» картинки.

Проблема: Ви супроводжуєте систему, одна із частин якої працює з зовнішнім сервісом підписання документів, наприклад, DocuSign. Періодично клієнти в вашій системі формують звіти в форматі pdf, далі ваша система відправляє їх на підпис в сервіс DocuSign і потім періодично перевіряє чи документ вже підписаний. Якщо документ підписаний, ви викачуєте його з сервісу і поміщаєте в своїй базі.

За останній рік кількість користувачів вашої системи виросло суттєво і почали приходити великі рахунки від DocuSign. Після аналізу ви розумієте, що рахунки зросли через велику кількість запитів з відправкою документів на підписання та запитів на перевірку чи документ вже підписаний. Після обговорення з бізнес-аналітиками та користувачам зрозуміли, що критичний інтервал доставки документів на підписання – 2 години і такий самий час критичності перевірки що документ підписаний і можна було б групувати всі

запит на відправку та на отримання і відправляти пакетом раз на годину. На даний момент клієнтський код працює з класом DocSignManager через інтерфейс IDocSignManager.

Рішення: Для вирішення проблеми можна застосувати патерн "Замісник". Ви реалізуєте клас замісник, який також реалізовує інтерфейс IDocSignManager, але він накопичує запити на відправку файлів на підписання і відправляє їх раз на годину, також він приблизно раз на годину отримує підписані документи, а на запити від клієнтів відповідає на основі інформації взятої з бази. Таким чином старий клас DocSignManager так само використовується для роботи з DocuSign сервісом, але вже набагато рідше, а клієнтський код взаємодіє з додатковим проміжним рівнем DocSignManagerProху, хоча з точки зору клієнтського коду нічого не змінилося і він працює з тим самим об'єктом IDocSignManager.

Реалізувавши такий підхід ви тепер економите 40% від попередньої вартості використання DocuSign сервіса і тепер основний вплив на вартість робить розмір файлів, що передаються на підпис, а не кількість запитів до служби.

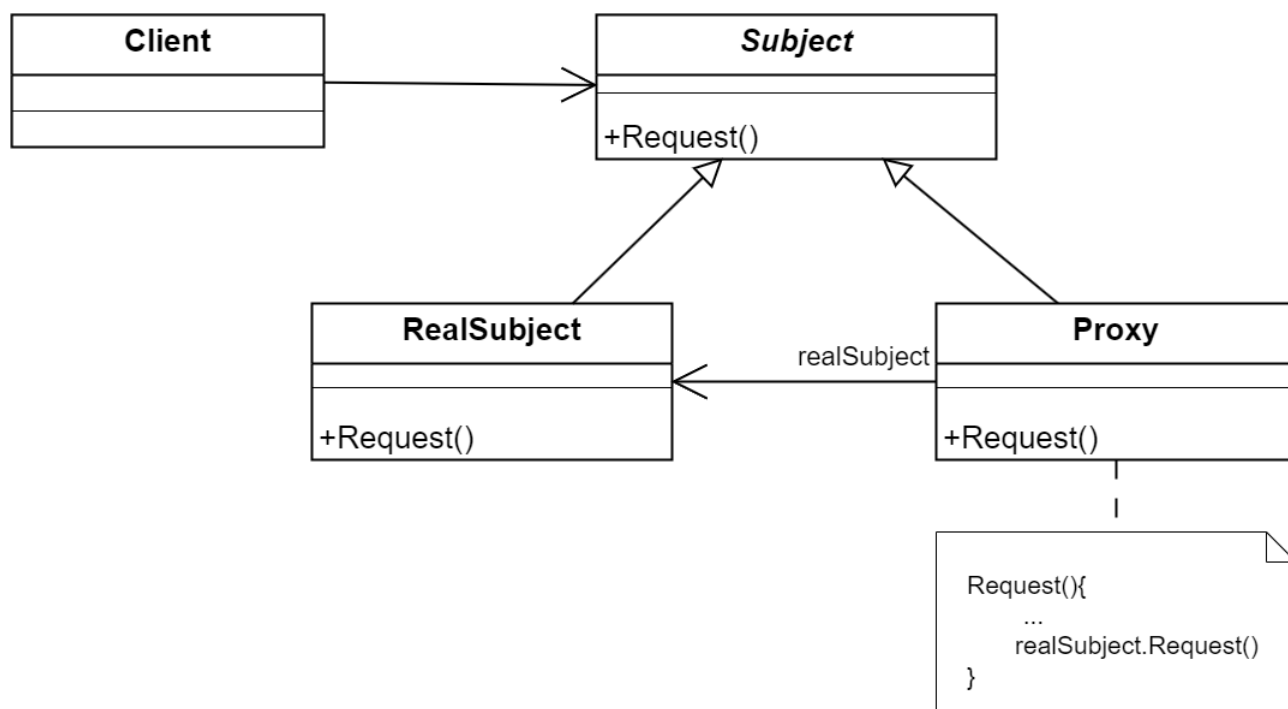


Рисунок 4.3. Структура патерну Proху

Переваги та недоліки:

- + Легкість впровадження проміжного рівня без переробки клієнтського коду.
- + Додаткові можливості по керуванню життєвим циклом об'єкту.
- Існує ризик падіння швидкості роботи через впровадження додаткових операцій.
- Існує ризик неадекватної заміни відповіді клієнтському коду

1.1.5. Шаблон «State»

Призначення: Шаблон «State» (Стан) дозволяє змінювати логіку роботи об'єктів у випадку зміни їх внутрішнього стану [6]. Наприклад, відсоток нарахованих на картковий рахунок грошей залежить від стану картки: Visa Electron, Classic, Platinum і т.д. Або обсяг послуг, які надані хостинг компанією, змінюється в залежності від обраного тарифного плану (стану членства – бронзовий, срібний або золотий клієнт). Реалізація даного шаблону полягає в наступному: пов'язані зі станом поля, властивості, методи і дії виносяться в окремий загальний інтерфейс (State); кожен стан являє собою окремий клас (ConcreteStateA, ConcreteStateB), які реалізують загальний інтерфейс [6, 8].

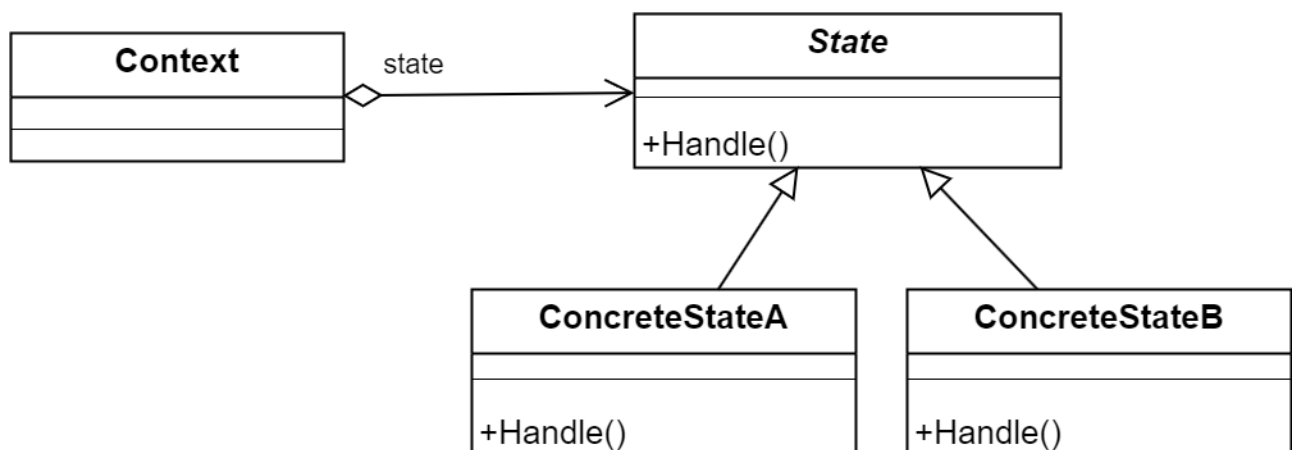


Рисунок 4.4. Структура патерну Стан

Об'єкти, що мають стан (Context), при зміні стану просто записують новий об'єкт в поле state, що призводить до повної зміни поведінки об'єкта.

Це дозволяє легко додавати в майбутньому і обробляти нові стани, відокремлювати залежні від стану елементи об'єкта в інших об'єктах, і відкрито проводити заміну стану (що має сенс у багатьох випадках).

Проблема: Ви розробляєте систему, яка складається з мобільних клієнтів та центрального сервера.

Для отримання запитів від клієнтів ви створюєте модуль Listener. Але вам потрібно щоб під час старту, поки сервер не запустився Listener не приймав запити від клієнтів, а після команди shutdown, поки сервер зупиняється, Listener вже не приймав запити від клієнтів, але відповіді, якщо вони будуть готові, відправив.

Рішення: Тут явно видно три стани для Listener з різною поведінкою: initializing, open, closing. Тому для вирішення краще за все підходить патерн State.

Визначаємо загальний інтерфейс IListenerState з методами, які по різному працюють в різних станах. Під кожний стан створюємо клас з реалізацією цього інтерфейсе. Контекст Listener при цьому всі виклики буде перенаправляти на об'єкт стану простим делегуванням. При старті він (Listener) буде посилатися на об'єкт стану InitializingState, знаходячись в якому Listener, фактично, буде ігнорувати всі вхідні запити. Після того як система запуститься і завантажить всі базові дані для роботи, Listener буде переключено в робочий стан, наприклад, викликом методу Open(). Після цього Listener буде посилатися на об'єкт OpenState і буде відпрацьовувати всі вхідні повідомлення в звичайному режимі. При запуску виключення системи, Listener буде переключено в стан ClosingState, викликом методу Close().

Переваги та недоліки:

- + Код специфічний для окремого стану реалізується в класі стану.
- + Класи та об'єкти станів можна використовувати з різними контекстами, за рахунок чого збільшується гнучкість системи.

- + Код контексту простіше читати, тому що вся залежна від станів логіка винесена в інші класи.
- + Відносно легко додавати нові стани, головне правильно змінити переходи між станами.
- Клас контекст стає складніше через ускладнений механізм переключення станів.

1.1.6. Шаблон «Strategy»

Призначення: Шаблон «Strategy» (Стратегія) дозволяє змінювати деякий алгоритм поведінки об'єкта іншим алгоритмом, що досягає ту ж мету іншим способом. Прикладом можуть служити алгоритми сортування: кожен алгоритм має власну реалізацію і визначений в окремому класі; вони можуть бути взаємозамінними в об'єкті, який їх використовує [6].

Даний шаблон дуже зручний у випадках, коли існують різні «політики» обробки даних. По суті, він дуже схожий на шаблон «State» (Стан), проте використовується в абсолютно інших цілях – незалежно від стану об'єкта відобразити різні можливі поведінки об'єкта (якими досягаються одні й ті самі або схожі цілі).

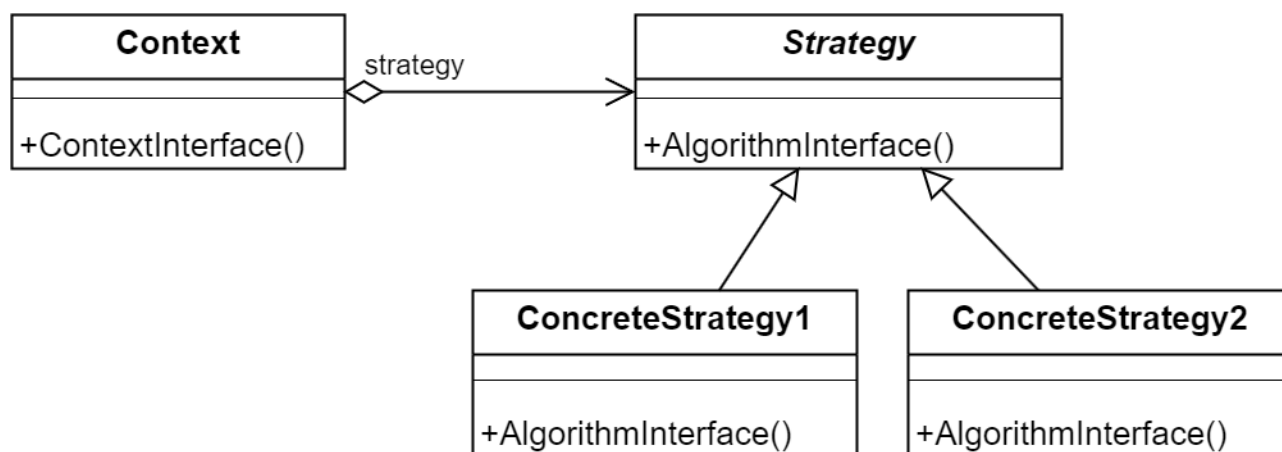


Рисунок 4.5. Структура патерну Стратегія.

Рішення: Коли ви використовуєте патерн «Стратегія», то схожі алгоритми виносяться з класа контекста в конкретні стратегії, за рахунок чого клас контексту стає чистіше і його легше супроводжувати. Також одні і тіж самі стратегії можна використати з різними контекстами, що значно збільшує гнучкість вашої системи та зменшує кількість дублювань у коді.

Контекст при цьому містить посилання на конкретну стратегію, а коли стратегію потрібно замінити, то замінюється об'єкт стратегії в полі `Context.strategy`.

Важливою умовою є відносна простота інтерфейсу для алгоритмів стратегій. Якщо алгоритмам стратегій прийдеться передавати десятки параметрів, то це, скоріш за все, приведе до ускладнення системи та заплутаності коду. Якщо стратегії на вході будуть приймати об'єкт контексту, щоб отримувати з нього всі необхідні дані, то такі стратегії будуть прив'язані до конкретного контексту і їх не можна буде використати з іншим типом контексту.

Приклад з життя: Ви їдете на роботу. Можна доїхати на автомобілі, на метро, або йти пішки. Тут алгоритм, як ви добираєтеся на роботу, є стратегією. В залежності від поточної ситуації ви вибираєте стратегію, що найбільше підходить в цій ситуації, наприклад, на дорогах великі пробки тоді ви їдете на метро, або метро тимчасово не ходить тоді ви їдете на таксі, або ви знаходитесь в 5 хвилинах ходьби від місця роботи і простіше добратися пішки.

Переваги та недоліки:

- + Використовувані алгоритми можна змінювати під час виконання.
- + Реалізація алгоритмів відокремлюється від коду, що його використовує.
- + Зменшує кількість умовних операторів типу `switch` та `if` в контексті.
- Надмірна складність, якщо у вас лише кілька невеликих алгоритмів.
- Під час виклику алгоритму, клієнтський код має враховувати різницю між стратегіями.

Хід роботи

1. Розробляю головний інтерфейс Archiver для всіх Strategy

```
beowulf-core / src / main / java / com / beowulf / core / strategy / Archiver.java / ...
You, last week | 1 author (You)
1 package com.beowulf.core.strategy;
2
3 import java.io.IOException;
4 import java.nio.file.Path;
5
You, 2 weeks ago | 1 author (You)
6 public interface Archiver {
7     /**
8      * Compresses the given source directory or file into a single archive file.
9      *
10     * @param sourceDir path to file or directory to compress
11     * @param targetArchive path to resulting archive file (.zip, .tar, etc.)
12     */
13     void compress(Path sourceDir, Path targetArchive) throws IOException;
14
15     /**
16      * Extracts the given archive file into the target directory.
17      *
18     * @param archive path to existing archive
19     * @param targetDir destination directory
20     */
21     void decompress(Path archive, Path targetDir) throws IOException;
22
23     String getName();
24 }
25 Ctrl+L to chat, Ctrl+K to generate
```

Рис. 1 Archiver інтерфейс

2. Розробляю ZipArchiver по шаблону Strategy застосовуючи попередньо створений ZipAdapter

```
1 package com.beowulf.core.strategy;
2
3 import com.beowulf.core.adapter.ZipAdapter;
4
5 import java.io.IOException;
6 import java.nio.file.Path;
7
8 public class ZipArchiver implements Archiver {
9
10     private final ZipAdapter adapter = new ZipAdapter();
11
12     @Override
13     public void compress(Path sourceDir, Path targetArchive) throws IOException {
14         adapter.compress(sourceDir, targetArchive);
15     }
16
17     @Override
18     public void decompress(Path archive, Path targetDir) throws IOException {
19         adapter.decompress(archive, targetDir);
20     }
21
22     @Override
23     public String getName() {
24         return "ZIP";
25     }
26 }
27
```

Рис. 2 ZipArchiver strategy

3. Розроблюю TarGzArchiver по шаблону Strategy застосовуючи попередньо створений TarGzAdapter

```
1 package com.beowulf.core.strategy; You, last week • ✨ Created TarGzAdapter ...
2
3 import com.beowulf.core.adapter.TarGzAdapter;
4
5 import java.io.IOException;
6 import java.nio.file.Path;
7
8 public class TarGzArchiver implements Archiver {
9
10     private final TarGzAdapter adapter = new TarGzAdapter();
11
12     @Override
13     public void compress(Path sourceDir, Path targetArchive) throws IOException {
14         adapter.compress(sourceDir, targetArchive);
15     }
16
17     @Override
18     public void decompress(Path archive, Path targetDir) throws IOException {
19         adapter.decompress(archive, targetDir);
20     }
21
22     @Override
23     public String getName() {
24         return "TAR";
25     }
26 }
27
```

Рис. 3 TarGzArchiver strategy

4. Розроблюю RarArchiver по шаблону Strategy застосовуючи попередньо створений RarAdapter

```
1 package com.beowulf.core.strategy; You, last week • ✨ Created RarArchiver str
2
3 import com.beowulf.core.adapter.RarAdapter;
4
5 import java.io.IOException;
6 import java.nio.file.Path;
7
8 public class RarArchiver implements Archiver {
9     private final RarAdapter adapter = new RarAdapter();
10
11     @Override
12     public void compress(Path sourceDir, Path targetArchive) throws IOException {
13         adapter.compress(sourceDir, targetArchive);
14     }
15
16     @Override
17     public void decompress(Path archive, Path targetDir) throws IOException {
18         adapter.decompress(archive, targetDir);
19     }
20
21     @Override
22     public String getName() {
23         return "RAR";
24     }
25 }
26
```

Рис. 4 RarArchiver strategy

5. Розроблюю AceArchiver по шаблону Strategy застосовуючи попередньо створений AceAdapter. Так як .ace формат є пропріетарним і належить GmbH, то єдине що ми можемо запропонувати користувачу це декомпресію використовуючи їхню офіційну бібліотеку

```
beowulf-core > src > main > java > com > beowulf > core > strategy > AceArchiver.java > ...

You, last week | 1 author (You)
1 package com.beowulf.core.strategy;
2
3 import java.io.IOException;
4 import java.nio.file.Path;
5
6 import com.beowulf.core.adapter.AceAdapter;
7
8 You, last week | 1 author (You)
9 public class AceArchiver implements Archiver {
10     private final AceAdapter adapter = new AceAdapter();
11
12     @Override
13     public void compress(Path sourceDir, Path targetArchive) {
14         throw new UnsupportedOperationException("ACE compression not supported.");
15     }
16
17     @Override
18     public void decompress(Path archive, Path targetDir) throws IOException {
19         adapter.decompress(archive, targetDir);
20     }
21
22     @Override
23     public String getName() {
24         return "ACE";
25     }
26 }
```

Рис. 5 AceArchiver strategy

6. Доповнюю діаграму класів

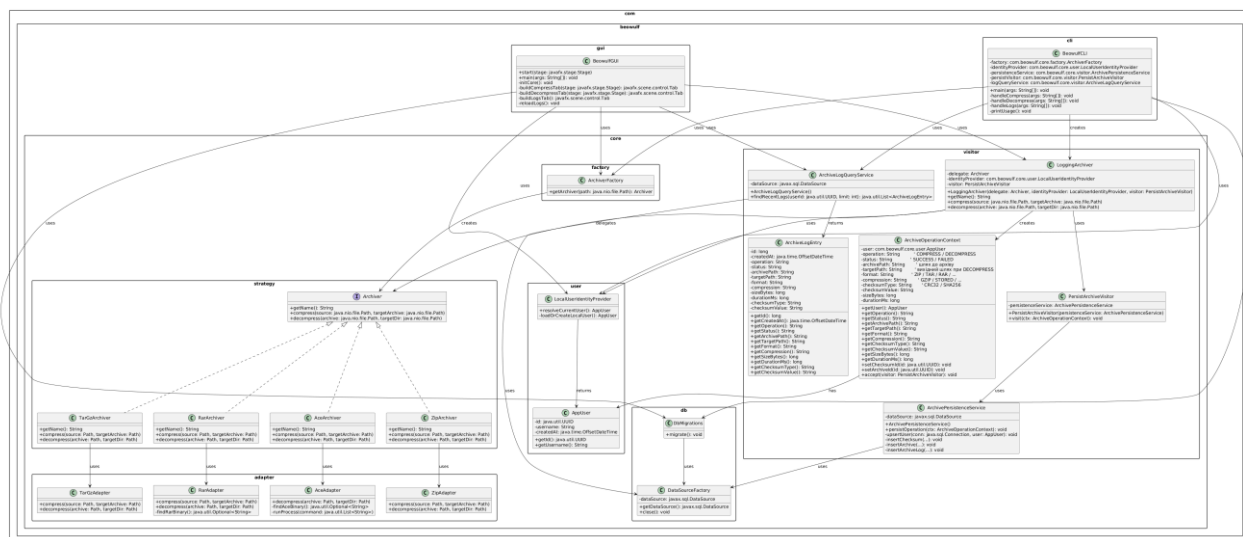


Рис. 6 Діаграма класів

Висновок

У цій лабораторній роботі я ознайомився з основними шаблонами проєктування (Singleton, Iterator, Proxy, State, Strategy) та детальніше попрацював із шаблоном Strategy на прикладі системи архіватора. Було спроектовано спільний інтерфейс Archiver і реалізовано окремі стратегії для ZIP, TAR.GZ, RAR та ACE з використанням відповідних адаптерів. Загалом робота продемонструвала практичну користь шаблонів проєктування для побудови гнучкої й масштабованої архітектури ПЗ.

Контрольні запитання

1. Що таке шаблон проєктування?

(Повторюване типове рішення поширеної задачі проєктування, оформлене як типовий опис структури класів/об'єктів і їхньої взаємодії.)

2. Навіщо використовувати шаблони проєктування?

(Щоб не вигадувати рішення з нуля, підвищити гнучкість і розширюваність коду, спростити підтримку та мати спільну «мову» між розробниками.)

3. Яке призначення шаблону «Стратегія»?

(Відокремити алгоритм від об'єкта-контексту і дозволити динамічно підміняти різні алгоритми, які досягають однієї цілі різними способами.)

4. Нарисуйте структуру шаблону «Стратегія».

(Абстрактний інтерфейс Strategy з методом execute(), кілька класів ConcreteStrategyA/B, що його реалізують, і клас Context, який має поле типу Strategy і викликає strategy.execute().)

5. Які класи входять в шаблон «Стратегія», та яка між ними взаємодія?

(Класи: Context, інтерфейс Strategy, конкретні стратегії ConcreteStrategyX. Контекст містить посилання на Strategy і делегує їй виконання алгоритму; конкретні стратегії реалізують різні варіанти цього алгоритму.)

6. Яке призначення шаблону «Стан»?

(Дозволити об'єкту змінювати свою поведінку залежно від внутрішнього стану, винісши логіку станів у окремі класи.)

7. Нарисуйте структуру шаблону «Стан».

(Інтерфейс State з методами поведінки, класи ConcreteStateA/B, що його реалізують, і клас Context, який має поле state: State і делегує виклики поточному стану; стани можуть змінювати контексту посилання на інший стан.)

8. Які класи входять в шаблон «Стан», та яка між ними взаємодія?

(Класи: Context, інтерфейс State, конкретні стани ConcreteStateX. Контекст зберігає поточний State і делегує йому операції; кожен ConcreteState реалізує поведінку для свого стану і може перевести контекст у інший стан.)

9. Яке призначення шаблону «Ітератор»?

(Надати уніфікований спосіб послідовного доступу до елементів колекції, не розкриваючи її внутрішню структуру.)

10. Нарисуйте структуру шаблону «Ітератор».

(Інтерфейс `Iterator` з методами `first()`, `next()`, `isDone`, `currentItem`, інтерфейс/клас `Aggregate` з методом `createIterator()`, конкретний `ConcreteIterator`, який ходить по `ConcreteAggregate` і зберігає поточну позицію.)

11. Які класи входять в шаблон «Ітератор», та яка між ними взаємодія?

(Класи: `Aggregate` / `ConcreteAggregate`, `Iterator` / `ConcreteIterator`. Колекція (`ConcreteAggregate`) створює свій `ConcreteIterator`, а клієнт працює з елементами через інтерфейс `Iterator`, не знаючи, як колекція реалізована всередині.)

12. В чому полягає ідея шаблону «Одинак»?

(Гарантувати, що у класу існує лише один екземпляр в системі, й надати до нього глобальну точку доступу через статичний метод/поле.)

13. Чому шаблон «Одинак» вважають «анти-шаблоном»?

(Бо він по суті створює глобальний стан, який важко тестувати й контролювати, порушує принцип єдиної відповідальності та часто маскує поганий дизайн.)

14. Яке призначення шаблону «Проксі»?

(Надати об'єкт-заступник, який контролює доступ до реального об'єкта, додаючи додаткову поведінку — кешування, ліниве завантаження, логування, контроль доступу, обмеження зовнішніх викликів тощо.)

15. Нарисуйте структуру шаблону «Проксі».

(Інтерфейс Subject, класи RealSubject і Proxy, які його реалізують; клієнт працює з Subject, а Proxy всередині делегує виклики до RealSubject, додаючи свій код до/після делегування.)

16. Які класи входять в шаблон «Проксі», та яка між ними взаємодія?

(Класи: Subject, RealSubject, Proxy. Клієнт звертається до Proxy через інтерфейс Subject, Proxy може виконати додаткові дії (перевірки, кешування, відкладені виклики) і потім викликає методи RealSubject.)